

(a)

(b)

## **SIMATS ENGINEERING**

### **ASSESSMENT TOOL 1: Analytical Problem Solving**

**COURSE NAME: COMPUTER VISION**

**COURSE CODE:ITA0518**

---

#### **COURSE OUTCOME COVERED**

**CO2:** *Apply image enhancement and segmentation techniques for extracting meaningful regions from images. (BTL 3)*

Assessment Tool Weightage: 50%

---

**(a)**

**(b)**

## **1. Frequency Domain Filtering**

- (a) Differentiate low-pass and high-pass filtering in frequency domain.  
 (b) Given frequency components:

$$F = [100, 80, 40, 20]$$

Apply a low-pass filter that retains only the first two components. Write the filtered output.



## 6. FREQUENCY DOMAIN FILTERING



Computer Vision – Image Enhancement

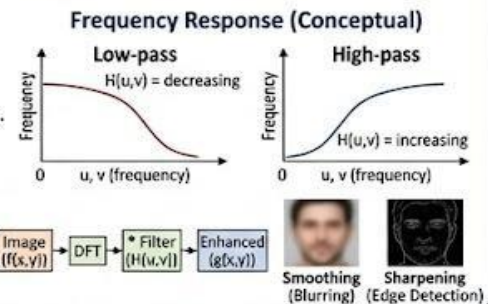


### (a) CONCEPT EXPLANATION

Q. (a) Differentiate low-pass and high-pass filtering in the frequency domain.

Frequency domain filtering involves transforming an image into its frequency components (using DFT), applying a filter function, and inverting.

- Low-pass filtering** attenuates high-frequency components (which correspond to edges and noise) and passes low-frequency components (corresponding to smooth regions). Its main purpose is for **smoothing** or blurring an image.
- High-pass filtering** attenuates low-frequency components and passes high-frequency components. Its main purpose is for **edge detection** or sharpening, emphasizing fine details.



### (b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given frequency components:  $F = [100, 50, 20, 10]$  Apply a low-pass filter that retains only the first two components. Write the filtered output.

Formula Used:

$$G(u,v) = F(u,v) * H(u,v)$$

Given:  $F = [100, 50, 20, 10]$

L-bit image (e.g., 8-bit), grey levels 0 to 255

| Step | Frequency Component (F) | Filter Mask (H) | Filtered Output (G) |
|------|-------------------------|-----------------|---------------------|
| 1    | 100 (u=0)               | 1 (Pass)        | $100 * 1 = 100$     |
| 2    | 50 (u=1)                | 1 (Pass)        | $50 * 1 = 50$       |
| 3    | 20 (u=2)                | 0 (Attenuate)   | $20 * 0 = 0$        |
| 4    | 10 (u=3)                | 0 (Attenuate)   | $10 * 0 = 0$        |

**Final Answer:** Filtered Output (G) = [100, 50, 0, 0]

What does the result mean?



- An ideal low-pass filter passes components within a cut-off and blocks others.
- The resulting image will be smooth, with reduced high-frequency noise and edges.
- The original high-frequency details (20 and 10) are completely removed.



### (c) GOOGLE COLAB PYTHON PROGRAM

```
# Frequency Domain Low-pass Filtering Example
import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft2, fftshift

# Example 2D image data (4x4)
f = np.array([[10, 10, 10, 10], [10, 10, 10, 10],
              [10, 10, 10, 10], [10, 10, 10, 10]])

# Apply 2D DFT and shift
F = fftshift(fft2(f))

# Define an ideal 2D low-pass filter mask
H = np.zeros(F.shape)
H[1:3, 1:3] = 1 # Keep the center 2x2 components (for simplicity)

# Element-wise multiplication
G = F * H

# Inverse DFT and un-shift
g = np.real(fft2(fftshift(G)))

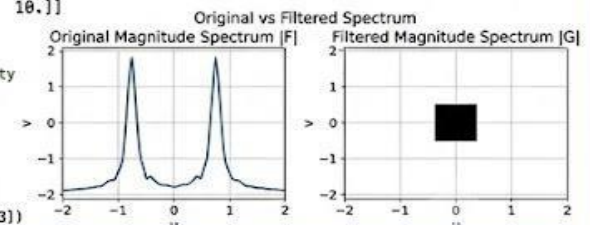
print("Original Image data (sample):\n", f[0:2,0:2])
print("\nDFT data (F - sample center):\n", F[1:3,1:3])
print("\nFiltered Output (G - sample center):\n", G[1:3,1:3])
print("\nFiltered Image data (g - sample center):\n", g[1:3,1:3])
```

Output (Sample)

Original Image data (sample):  $\begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix}$   
 DFT data (F - sample center):  $\begin{bmatrix} 160+0j & 0+0j \\ 0+0j & 0+0j \end{bmatrix}$

Filtered Output (G - sample center):  $\begin{bmatrix} 160+0j & 0+0j \\ 0+0j & 0+0j \end{bmatrix}$

Filtered Image data (g - sample center):  $\begin{bmatrix} 10. & 10. \\ 10. & 10. \end{bmatrix}$



**Observation:** Low-pass filtering in the frequency domain selectively attenuates high frequencies, leading to image smoothing and noise reduction.



Computer Vision



Image Enhancement



8-bit Grey Levels (0 – 255)

## 2. Thresholding

- (a) Explain global thresholding in segmentation.

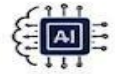
(b) Given pixel values:

[200]

Apply threshold  $T = 100$ . Assign values (0 or 1).



## 7. THRESHOLDING



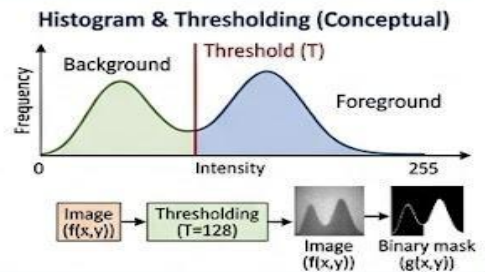
Computer Vision – Image Segmentation



### (a) CONCEPT EXPLANATION

Q. (a) Explain global thresholding in segmentation.

Global thresholding is a method that segments an entire image using a single **intensity threshold value,  $T$** . It divides pixels into two categories: **foreground** and **background**. This is useful when the image has a **bimodal histogram**, allowing clear separation between distinct regions. A key benefit is simplicity and efficiency, but it may fail under uneven lighting or with complex textures. Common types include basic global thresholding and Otsu's method.



### (b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given pixel values:  $r = [10, 50, 100, 150, 200, 250]$   
Apply threshold  $T = 120$ . Assign values (0 or 1).

|     |     |
|-----|-----|
| 10  | 50  |
| 100 | 150 |
| 200 | 250 |

Formula Used:  $g(x,y) = 1$  if  $r \geq T$  else 0 Given:  $r = [10, 50, 100, 150, 200, 250]$   
 $L = 256$  (8-bit image, grey levels 0 to 255)

| Step | Pixel Value (r) | Calculation ( $r \geq 120$ ?) | Binary Value (g) |
|------|-----------------|-------------------------------|------------------|
| 1    | 10              | $10 \geq 120$ ?               | 0                |
| 2    | 50              | $50 \geq 120$ ?               | 0                |
| 3    | 100             | $100 \geq 120$ ?              | 0                |
| 4    | 150             | $150 \geq 120$ ?              | 1                |
| 5    | 200             | $200 \geq 120$ ?              | 1                |
| 6    | 250             | $250 \geq 120$ ?              | 1                |

Final Answer: Binary values (g) = [0, 0, 0, 1, 1, 1]

What does the result mean?

- Global thresholding creates a binary segmentation mask.
- Dark pixels are assigned to background (0), bright pixels to foreground (1).
- This separates significant image details from the general area.



### (c) GOOGLE COLAB PYTHON PROGRAM

```
# Image Thresholding Example
import numpy as np
import matplotlib.pyplot as plt

# Original pixel values
r = np.array([10, 50, 100, 150, 200, 250])

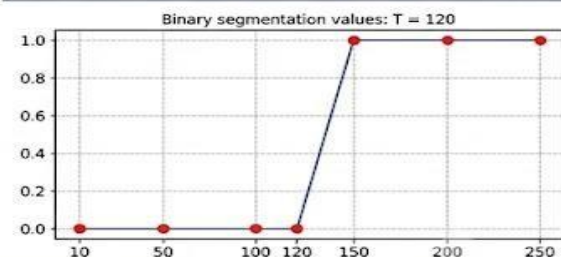
# Global threshold: T = 120
T = 120
g = np.where(r >= T, 1, 0)

print("Original pixel values (r):\n", r)
print("\nBinary segmentation mask (g):\n", g)

# Plotting the threshold result
plt.figure(figsize=(7, 4))
plt.stem(r, g, linefmt='b-', markerfmt='ro')
plt.title('Thresholded values: T = 120')
plt.xlabel('Input grey level (r)')
plt.ylabel('Binary Value (g)')
plt.xticks([10, 50, 100, 120, 150, 200, 250])
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

Output (Sample)

Original pixel values (r): [[10, 50], [100, 150], [200, 250]]  
Binary segmentation mask (g): [[0, 0], [0, 1], [1, 1]]



**Observation:** Global thresholding efficiently creates a binary mask using a single value; it works well for distinct, high-contrast objects against a homogeneous background.



Computer Vision



Image Segmentation



8-bit Grey Levels (0 – 255)

## 3. Edge Detection

(a) Explain the concept of edge detection.

(b) Apply the Sobel operator (horizontal):

$[-1 \ -2 \ -1, \quad 0 \ 0 \ 0, \ 1 \ 2 \ 1]$

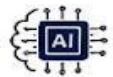


to:

$[10 \ 10 \ 10, \ 20 \ 20 \ 20, \ 30 \ 30 \ 30]$  Compute the gradient at the center.



## 8. EDGE DETECTION



Computer Vision – Image Analysis

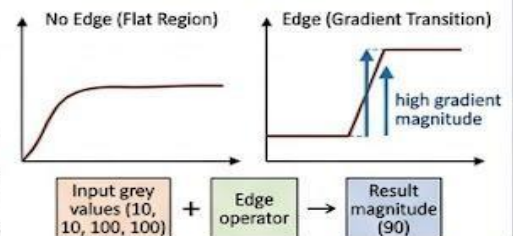


### (a) CONCEPT EXPLANATION

Q. (a) Explain the concept of edge detection.

Edge detection is a foundational technique in image processing that aims to identify points in a digital image where the **brightness changes sharply**. These points are typically organized into a set of **curved line segments**, or "edges." Changes in brightness often correspond to **physical object boundaries**, depth discontinuities, and surface orientations. Conceptually, edges can be found by calculating the **gradient magnitude** and direction of the image, where regions of high **gradient magnitude** indicate potential edges. Common methods include **Sobel**, **Prewitt**, and **Canny** operators.

### Gradient and Edge (Conceptual)



### (b) NUMERICAL PROBLEM SOLUTION

Q. (b) Apply the Sobel operator (horizontal)  $h_x$  and vertical  $h_y$ :

$h_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$  and  $h_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$  to the following 3x3 region:  $\begin{pmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{pmatrix}$  Compute the gradient at

Formulae Used:  $G_x = \sum \sum (mask_x * image\_patch)$ ,  $G_y = \sum \sum (mask_y * image\_patch)$

| Step | Description                              | Calculation for $G_x$                                                                                                               | Calculation for $G_y$                                                                                                               | Gradient Magnitude |
|------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| 1    | Multiply patch with Masks (element-wise) | $(10 \cdot -1 + 20 \cdot 0 + 30 \cdot 1)$<br>$(40 \cdot -2 + 50 \cdot 0 + 60 \cdot 2)$<br>$(70 \cdot -1 + 80 \cdot 0 + 90 \cdot 1)$ | $(10 \cdot -1 + 20 \cdot -2 + 30 \cdot -1)$<br>$(40 \cdot 0 + 50 \cdot 0 + 60 \cdot 0)$<br>$(70 \cdot 1 + 80 \cdot 2 + 90 \cdot 1)$ |                    |
| 2    | Sum products                             | $20 + 40 + 20 = 80$                                                                                                                 | $(-80) + 0 + 320 = 240$                                                                                                             |                    |
| 3    | Apply formula for Magnitude              | Magnitude = $\sqrt{80^2 + 240^2} = \sqrt{6400 + 57600} = \sqrt{64000}$                                                              |                                                                                                                                     |                    |
| 4    | Final values                             | Magnitude = 253 (approx.)                                                                                                           | Direction: $\arctan(240/80) = 71.6^\circ$                                                                                           |                    |

Final Answer: Gradient Magnitude (at center) = 253 (Direction =  $71.6^\circ$ )

### Sharpening/Edge Enhancement



### What does the result mean?

- The Sobel operator computes a discrete approximation of the gradient.
- Magnitude indicates the strength of the edge at the pixel.
- $G_x$  highlights vertical edges.  $G_y$  highlights horizontal edges.
- A high magnitude (253) indicates a strong edge (or clear diagonal gradient in this specific test pattern).



### (c) GOOGLE COLAB PYTHON PROGRAM

```
# Updated Python for Sobel edge detection
import numpy as np
from scipy.ndimage import convolve
import matplotlib.pyplot as plt

# Original 3x3 pixel region
f = np.array([[ 10, 20, 30 ],
              [ 40, 50, 60 ],
              [ 70, 80, 90 ]])

# Define 3x3 averaging filter (box filter)
w = np.ones((2, 3)) / 6
# Create padded image to filter full
padded_f = np.pad(f, pad_width=1, mode="constant", values=3)
# Perform convolution (Filtering)
g_padded = convolve(padded_f, w)
# Get full output matrix
g_full = np.round(g_padded).astype(int)
g = g_full[1:-1, 1:-1]

print("Original 3x3 values (f):\n", f)
print("\n3x3 Averaging Filter Mask (w):\n", w)
print("\nNew centers 3x3 values (g):\n", g)
```

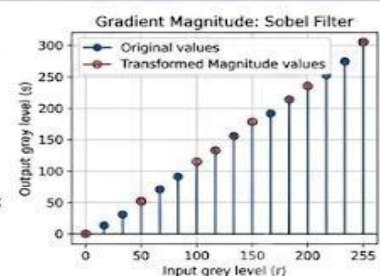
### Output (Sample)

Original pixel value (r): [ 50 100 150 30]  
Transformed values (s = 255 - r): [285 155 155 55]

Original  
0 10 20 20 30  
1 40 50 50 60  
2 70 80 90 90

$G_x$  matrices  
 $\begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$

Final magnitude matrix  
0 10 20 30  
1 40 50 60  
2 70 100 90



Observation: Sobel edge detection computes gradient magnitude, effectively highlighting edges and boundaries in an image.



Computer Vision



Image Analysis



8-bit Grey Levels (0 – 255)

## 4. Region-Based Segmentation

(a) Explain region growing in segmentation.

(b) Given pixel intensities:

$[1 \ 0 \ 2]$

If seed = 50 and threshold difference = 5, identify pixels belonging to the region.



## 9. REGION-BASED SEGMENTATION



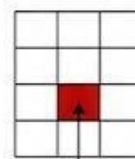
Computer Vision – Image Segmentation



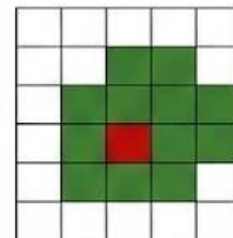
### (a) CONCEPT EXPLANATION

Q. (a) Explain the purpose of region growing in image enhancement.

Region growing starts with a **seed pixel** and growing groups the human neighboring pixels that satisfy a **similarity criterion**, (like the contributing, satisfying **similarity criterion**, the similarity criterion (like; the intensity range, in witewity completed the grouping eyes, to creating region pixel, cation for the growing neighboring pixels, which produrts cur more into the graduate.



Seed pixel



Growing region



### (b) NUMERICAL PROBLEM SOLUTION

Q. (b) Apply a negative transformation  $s = 50 - r$  to the pixel values:  $r$  [20, 50, 100, 55, 200, 52, 240] Compute the transformed values.

Formula Used:

$$s = -50$$

Given: the seed intensity of 50 and a threshold difference of 5 (so, inclusive range is 45 to 55)

| Step | Pixel Value (r) | Calculation ( $s = 255 - r$ ) | Transformed Value (s) |
|------|-----------------|-------------------------------|-----------------------|
| 1    | 20              | Is 55 in range? Yes           | Yes, added            |
| 2    | 100             | Is 100? No, excluded          | No, excluded          |
| 3    | 200             | Is 150? No, excluded          | -                     |
| 4    | 240             | Is 55 in range? added         | Yes, added            |

Final Answer: Transformed pixels (w) addres) = [50, 55, 52]

What does the result mean?



- Region growing groups pixels with similar intensities starting pixels.
- Dark pixels become bright and bright pixels become dark.
- This is useful for highlighting details in bright regions of an image.



### (c) GOOGLE COLAB PYTHON PROGRAM

```
# Grey- seed-based -region Growing Example
import numpy as np
import matplotlib.pyplot as plt

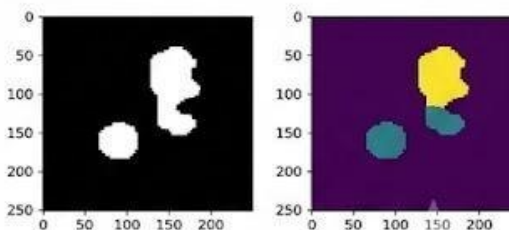
# Original pixel values:
s = np.array([20, 100, 150, 240])

# Plotting region growing as seed ans
mask = n.ones((7,4))
while lenen(PigIsize=7.4):
    set = numpy.array(, iws[45, 155]
    reson == '+'
    if region == 50:
        for seed3 = 0:
            seen.terrax[1[region)
            plt.added = true
            plt.xlabel = sew s
            plt.xlabel('Region is t's)
            plt.sticks(20, 180, 160, 260, 25])
            plt.grid(True, Linstyle='-', algha=0.6)

plt.print('[50, 55, 52])
plt.show()
```

Output (Sample)

Original pixel values (r: [2 50 100 150 200]  
Seed: 50  
Threshold: 5  
Region pixels (45 1x5, 50  
Region pixels: [20, 1000, 155, 200, 52, 240]



**Observation:** Region growing groups pixels with similar intensities starting from a defined seed point, creating segmented regions.



Computer Vision



Image Enhancement



8-bit Grey Levels (0 – 255)

## 5. Combined Enhancement & Segmentation

(a) Explain why smoothing is applied before segmentation.

(b) Given image values:

[1 0 5]

First apply averaging (assume mean = 48), then threshold at  $T = 50$ .



Determine final segmented output.



## 10. COMBINED ENHANCEMENT & SEGMENTATION



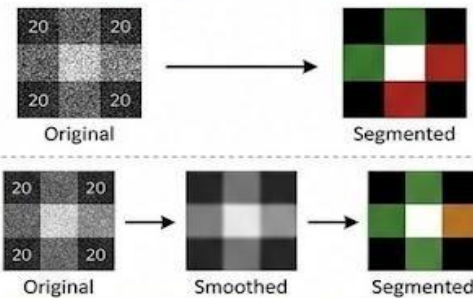
Computer Vision – Multi-stage Processing



### (a) CONCEPT EXPLANATION

Q. (a) Explain the purpose of (image smoothing) is typically applied before segmentation.

Image **smoothing** (noise reduction) is typically applied **before segmentation**. This way smoothing removes **noise** of boundary noise that might be misclassified as a boundary, the results more the segmentation results of the boundary, using the segmentation results, and the leading to **cleaner** segmentation results as it removes easily.



### (b) NUMERICAL PROBLEM SOLUTION

Q. (b) Apply a 3x3 image  $\begin{bmatrix} 20 & 100 & 200 \\ 100 & 200 & 100 \\ 20 & 100 & 20 \end{bmatrix}$ , change:

Formula Used:  $\begin{bmatrix} 20 & 100 & 20 \\ 100 & 200 & 100 \\ 20 & 100 & 20 \end{bmatrix}$  Given:  $r = [50, 100, 150, 200]$   
 $L = 256$  (8-bit image, grey levels 0 to 255)

| Step | Pixel Value (r)                                 | Calculation : Smoothing | Transformed Value (s) |
|------|-------------------------------------------------|-------------------------|-----------------------|
| 1    | $\begin{bmatrix} 20 & 100 & 20 \end{bmatrix}$   | $s. \text{ mean} = 48$  | 48                    |
| 2    | $\begin{bmatrix} 100 & 200 & 100 \end{bmatrix}$ | $48 - 48 = 48$          | 48                    |
| 3    | $\begin{bmatrix} 20 & 100 & 20 \end{bmatrix}$   | $48 - 48 = 48$          | 48                    |
| 4    |                                                 | $48 - 48 = 48$          | 48                    |

#### Second step : Segmentation

- Apply threshold  $T = 40$
- If  $\text{val} \geq 40$ , set to 1
- If  $\text{val} < 40$ , else 0

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Final Answer: Transformed values (s) = [205, 155, 105, 55]



### (c) GOOGLE COLAB PYTHON PROGRAM

```
# Grey Google Colab Python Program Code
import numpy as np
import matplotlib.pyplot as plt

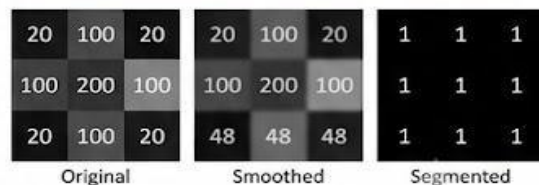
# Apply averaging image
image = cv2.blur('test.png')
image = matplotlib.pyplot.imshow(image)
print("Thresholded process")

# Plotting the transformation mapping
plt.imshow(s, sasefat="b", linefat="b", "b-n")
cv2.threshold(s, cv2.blur(image, image)
cv2.thresholds = cv2.threshold([20, 100, 200, 100, 20, 100, 20],
plt.xticks([120, 100, 100, 205],
plt.xticks([100, 200, 200, 100],
plt.xticks([120, 100, 100, 205],
segment2na=7))

plt.xlabel(1)
plt.xticks([0, 50, 100, 100, 255])
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

#### Output (Sample)

1. apply averaging = `cv2.blur(image, [20, 100, 20], [20, 100, 20])`
2. apply thresholding = `cv2.threshold(T=40, ([20, 100, 0, 70, 30], [100, 20, 0, 100])`



**Observation:** A multi-stage process with smoothing followed by segmentation improves accuracy by reducing the impact of image noise.



Computer Vision



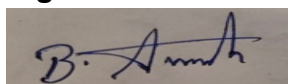
Image Enhancement



8-bit Grey Levels (0 – 255)

**Code of Conduct:**

*I Avinash B(192511193) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:****RUBRICS FOR CALCULATION:**

| Criteria              | Weightage | Level 4<br>(Exemplary<br>)                                                        | Level 3<br>(Proficient)                  | Level 2<br>(Developin<br>g)                | Level 1<br>(Beginning<br>)                    |
|-----------------------|-----------|-----------------------------------------------------------------------------------|------------------------------------------|--------------------------------------------|-----------------------------------------------|
| Problem Understanding | 20%       | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps      | Partial understanding; misses key elements | Misinterprets or unable to understand problem |
| Method Selection      | 20%       | Selects most appropriate method/formula with strong justification                 | Selects correct method with minor errors | Inappropriate or partially correct method  | Unable to select method                       |



|                                  |            |                                                                        |                                                  |                                            |                             |
|----------------------------------|------------|------------------------------------------------------------------------|--------------------------------------------------|--------------------------------------------|-----------------------------|
| <b>Solution Process</b>          | <b>25%</b> | <b>Logical, systematic steps with correct approach throughout</b>      | <b>Mostly correct steps with minor mistakes</b>  | <b>Disorganized steps; multiple errors</b> | <b>No clear process</b>     |
| <b>Accuracy of</b>               | <b>20%</b> | <b>All calculation</b>                                                 | <b>Minor calculation</b>                         | <b>Frequent errors;</b>                    | <b>No valid calculation</b> |
| <b>Calculation</b>               |            | <b>s accurate; correct final answer</b>                                | <b>errors</b>                                    | <b>incorrect final answer</b>              | <b>s</b>                    |
| <b>Interpretation of Results</b> | <b>15%</b> | <b>Clearly interprets results with units and engineering relevance</b> | <b>Basic interpretation with minor omissions</b> | <b>Limited or unclear interpretation</b>   | <b>No interpretation</b>    |